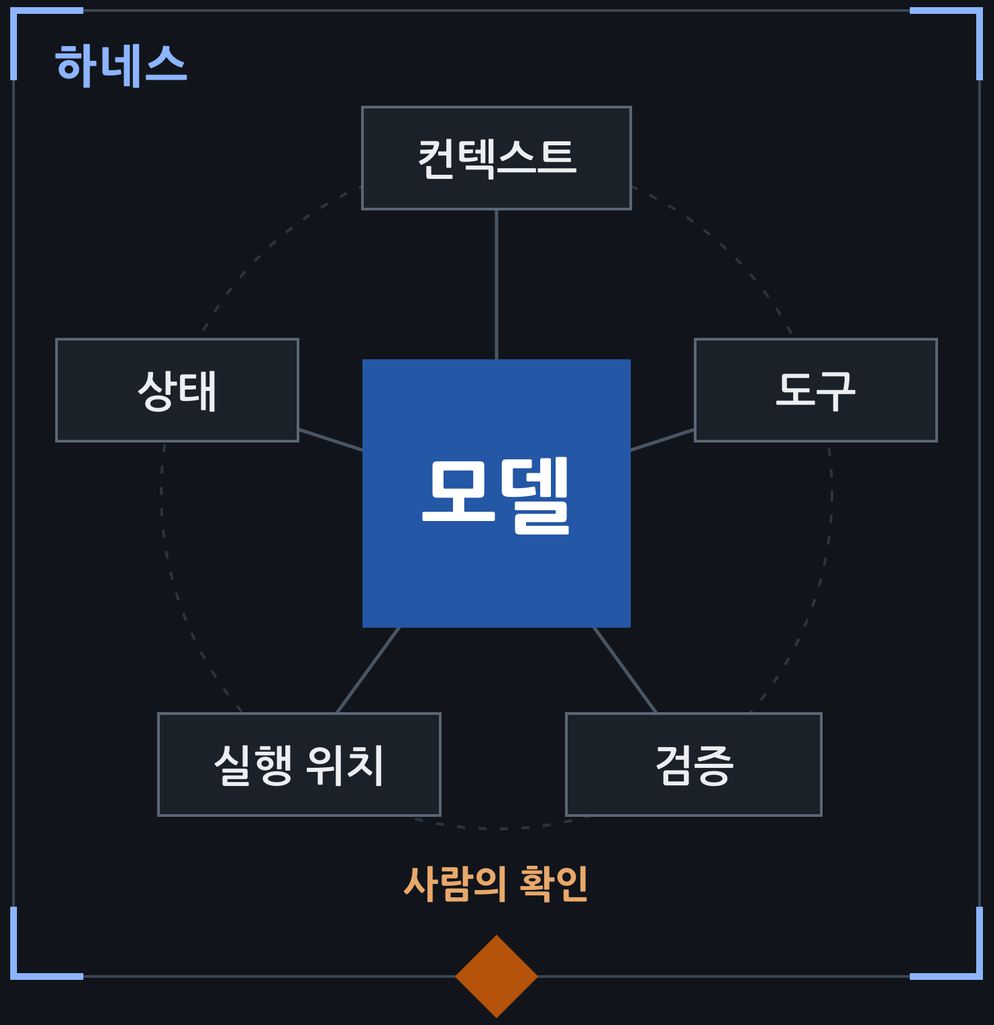
120분 워크숍

하네스 엔지니어링

AI에게 일을 맡기는 구조를 설계하는 법

용어 25개와 사례 5개로 읽는 작업의 틀

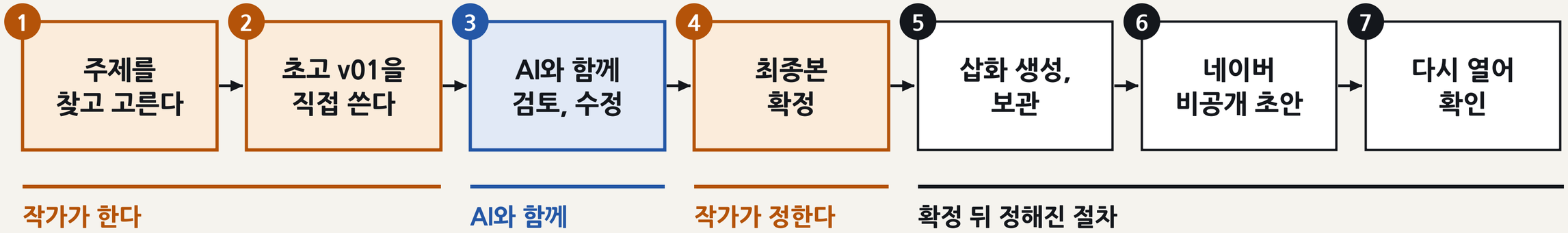


시작

완성된 업무 한 건에서 출발한다

AI교양 글 한 편이 끝났다. 작가가 주제를 고르고 초고를 직접 썼다. AI와 함께 다듬은 뒤 작가가 확정했고, 그다음 삽화와 비공개 초안을 만들어 다시 열어 확인했다.

v01은 작가가 직접 쓴 원고다. AI가 만든 초안이 아니다.



이 흐름이 단계를 건너뛰지 않게 하는 것은 누구일까?

설명용 정리다. 실제 화면, 계정 정보, 공개 발행은 보여 주지 않는다. 낯선 라벨은 뒤에서 하나씩 설명한다.

오늘의 길

다섯 개의 질문을 따라간다

01

AI는 무엇을 하고, 누가 실행할까?

모델, 에이전트, 런타임, 하네스

사례 OpenClaw
18분

02

일하는 방법을 어디에 담을까?

오케스트레이션, 도구, 스킬, 플러그인, 커넥터, MCP

사례 OMH, Grokbot
25분

03

AI는 무엇을 알고 있을까?

프롬프트, 컨텍스트, 토큰, 컨텍스트 창, 메모리

사례 AI교양
20분

휴식 10분

04

누가 이어서 일할까?

세션, 워커, 서브에이전트

사례 CEO Agent
13분

05

끝났다는 말을 어떻게 믿을까?

워크플로우, 자동화, 완료 보고, 검증, 로컬 실행, 클라우드 실행, 리모트 컨트롤

사례 종합
24분

개념과 사례를 번갈아 본다. 아직 배우지 않은 용어는 사례에서 이름만 보이고, 해당 슬라이드에서 설명한다.

01

1장 약 18분

AI는 무엇을 하고, 누가 실행할까?

이 장을 마치면 모델이 하는 일과 실행하는 쪽을 나눠 말할 수 있다. 네 개념을 익힌 뒤 OpenClaw 구조에서 직접 찾는다.

모델 — 판단과 생성

에이전트 — 반복과 종료

런타임 — 실제 실행

하네스 — 작업 틀 전체

사례 OpenClaw — 구조 안에서 찾기

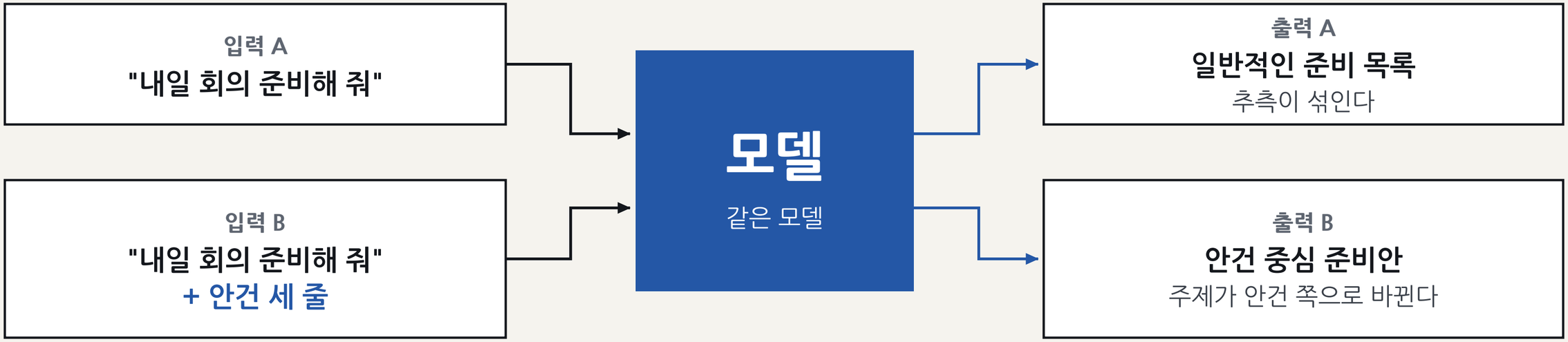
모델

장면

"내일 회의 준비해 줘." 안건도 참석자도 없다.

그럴듯한 준비안이 나올까?

입력된 텍스트와 맥락을 받아 다음 출력을 만드는 학습된 계산 장치.
출력에는 답변과 도구 호출 요청이 들어갈 수 있다.



출력에는 도구 호출 요청도 있다 `read_file(path=tickets.csv)` 요청만 나온다. 파일은 여기서 열리지 않는다.

헷갈리기 쉬운 점 모델은 출력을 만든다. 출력이 도구 실행과 결과 확인으로 이어져 여러 단계를 도는 구성은 에이전트다.

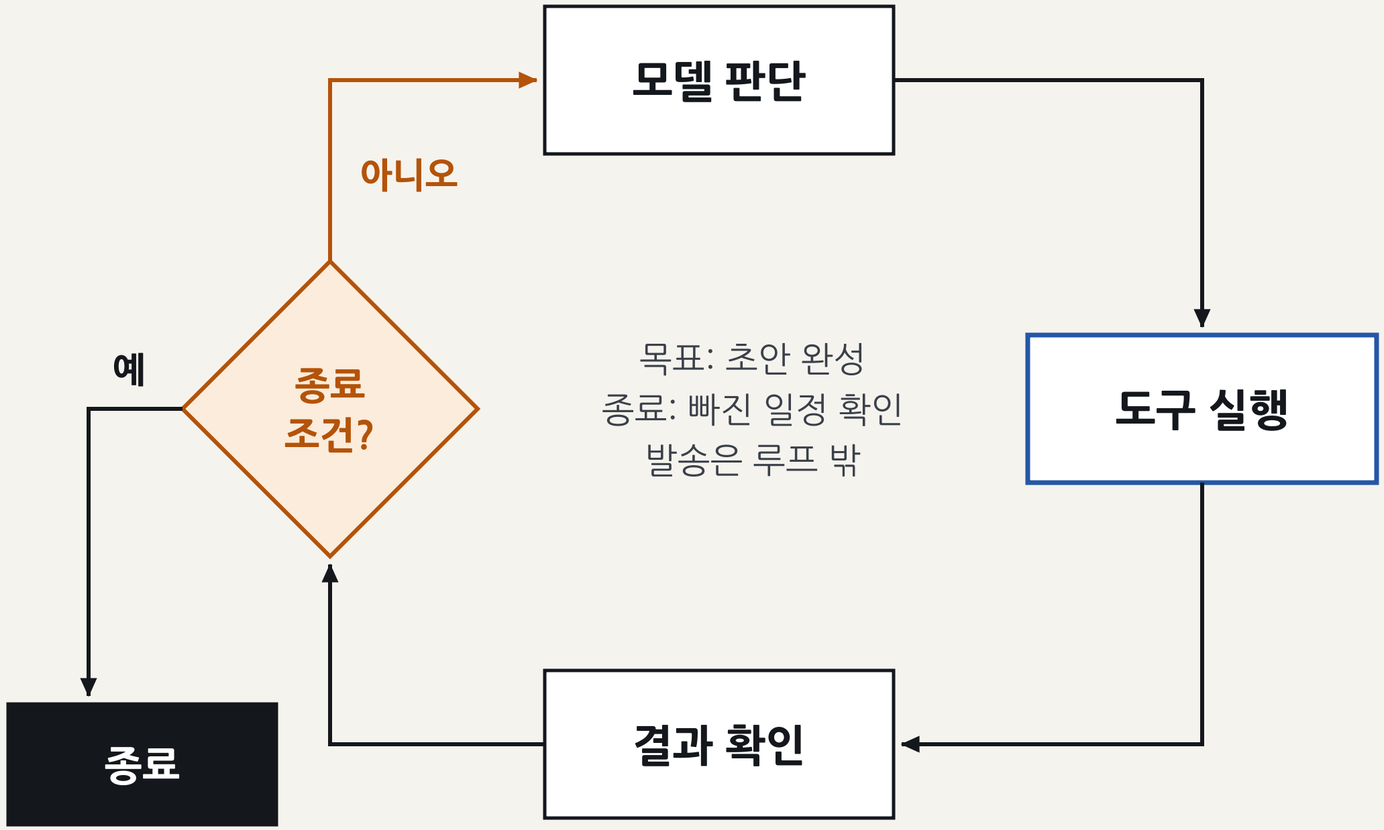
에이전트

목표를 받아 모델의 판단, 도구 실행, 결과 확인을 반복하고, 종료 조건이 충족되면 멈추는 실행 구성.

장면

일정 안내 메일 초안을 맡긴다. 빠진 일정은 되물어 채우고 고친다. 발송은 하지 않는다.

끝나는 조건이 있는 반복이면 에이전트일까?



헷갈리기 쉬운 점 응답 횟수로 판단하지 않는다. 결과를 보고 다음 행동이나 종료를 정하는 구조인지 본다.

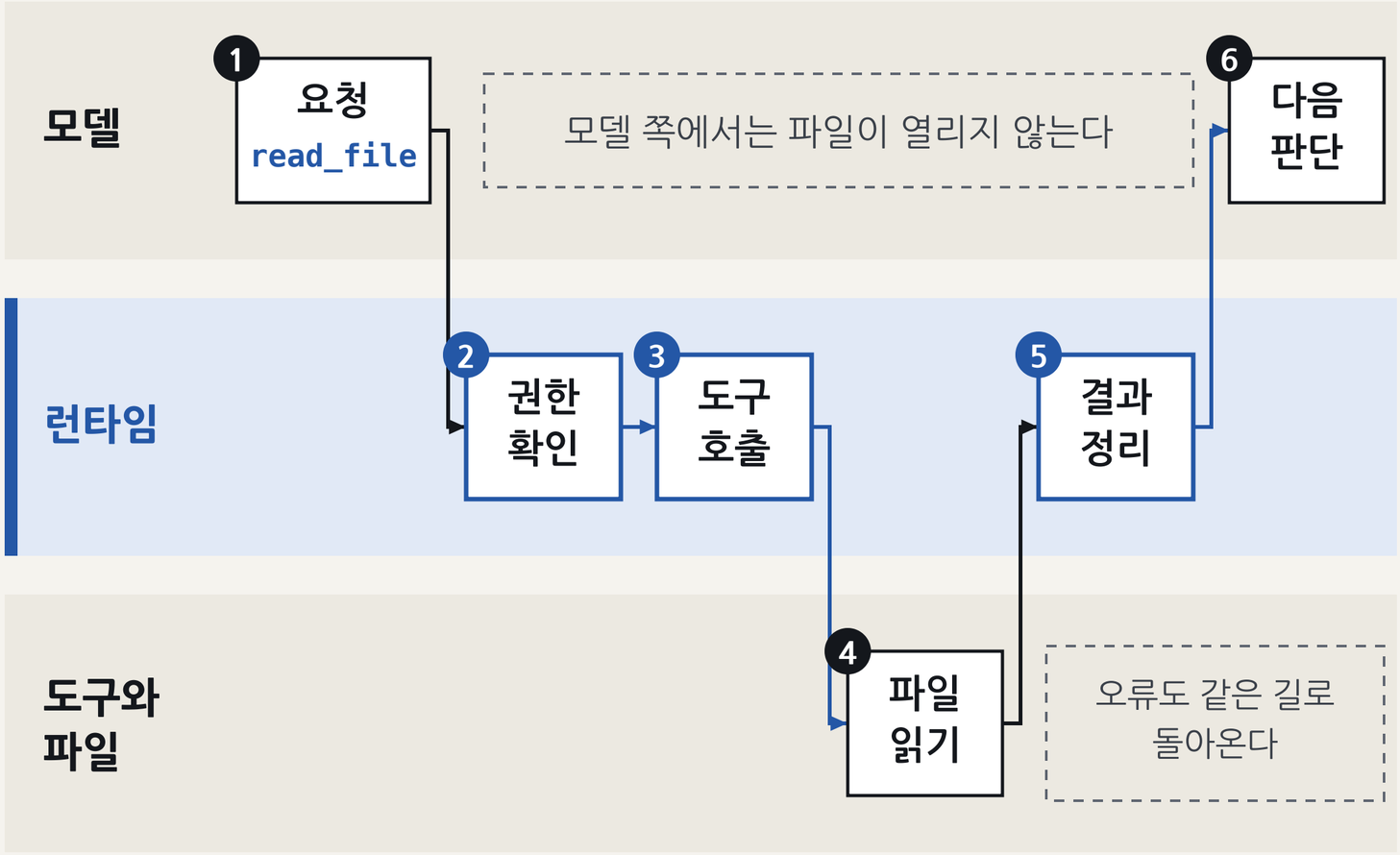
런타임

모델의 도구 호출 요청을 받아 권한을 확인하고 도구를 실행한 뒤, 결과를 다시 모델 입력으로 돌려주는 실행 계층.

장면

"지난주 회의 메모 파일 열어 줘."

실제로 파일을 여는 쪽은 누구일까?



헷갈리기 쉬운 점 Python, Node.js 같은 언어 실행 환경과 다르다. 하네스보다 좁은, 실행을 잇는 층이다.

하네스

장면

검토자가 체크리스트 파일을 폴더에 넣고 검토를 시작한다.

파일이 폴더에 있으면 검토 틀이 갖춰진 걸까?

모델이 안정적으로 일하도록 컨텍스트, 도구와 실행 위치, 상태 저장, 검증 절차를 묶은 작업 틀 전체. 이 워크숍의 정의이며 제품마다 범위가 다를 수 있다.

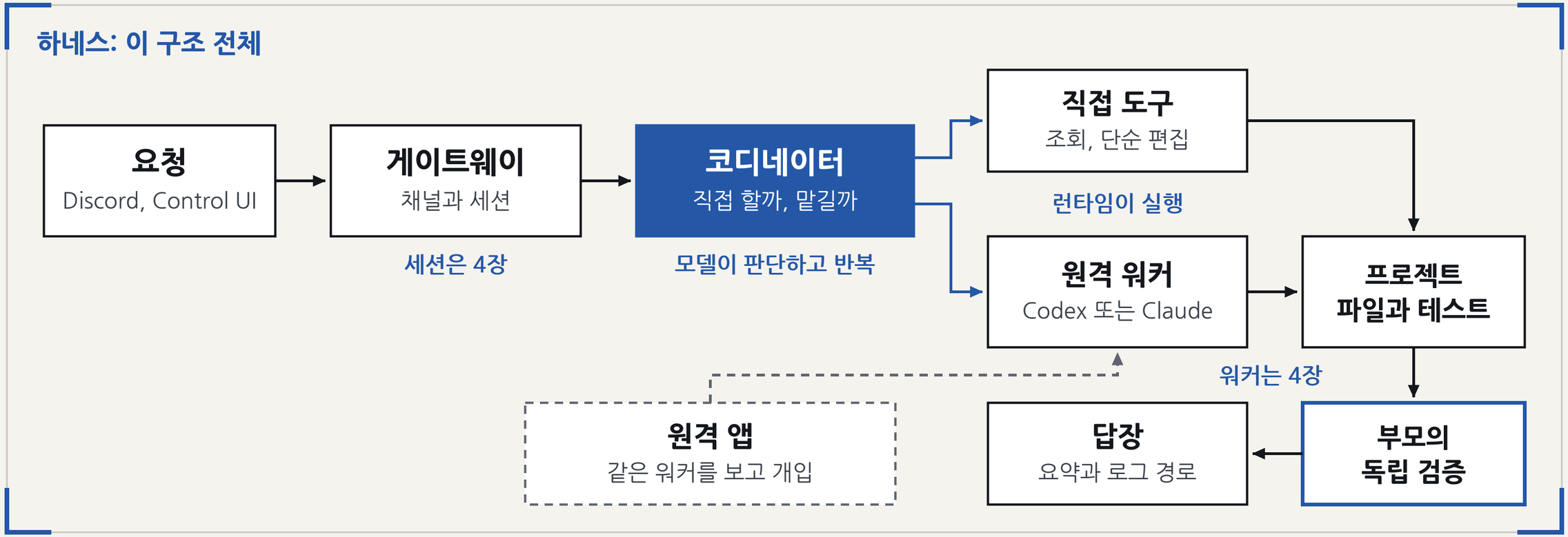


헷갈리기 쉬운 점 하네스는 틀이고, 오케스트레이션은 그 안의 순서와 역할 배정이다. 런타임은 실행을 잇는 층이다.

사례 1 / 5 OpenClaw

질문 디스코드에서 "보고서 검증해 줘"가 왔다. 받는 쪽, 맡기는 쪽, 확인하는 쪽은 누구일까?

요청을 받고, 맡기고, 따로 확인하는 작업 구조



운영 문서 기준의 가상 예시이며 실시간 상태는 확인하지 않았다. 위임 기준은 운영 지침이지 자동 라우터가 아니다. 모델 추론 위치는 따로 본다.

02

2장 약 25분

일하는 방법을 어디에 담을까?

역할과 순서, 실행되는 기능, 다시 쓰는 절차를 나눈다. 그다음 설치와 연결을 맡는 부품을 구분하고, 두 사례에서 확인한다.



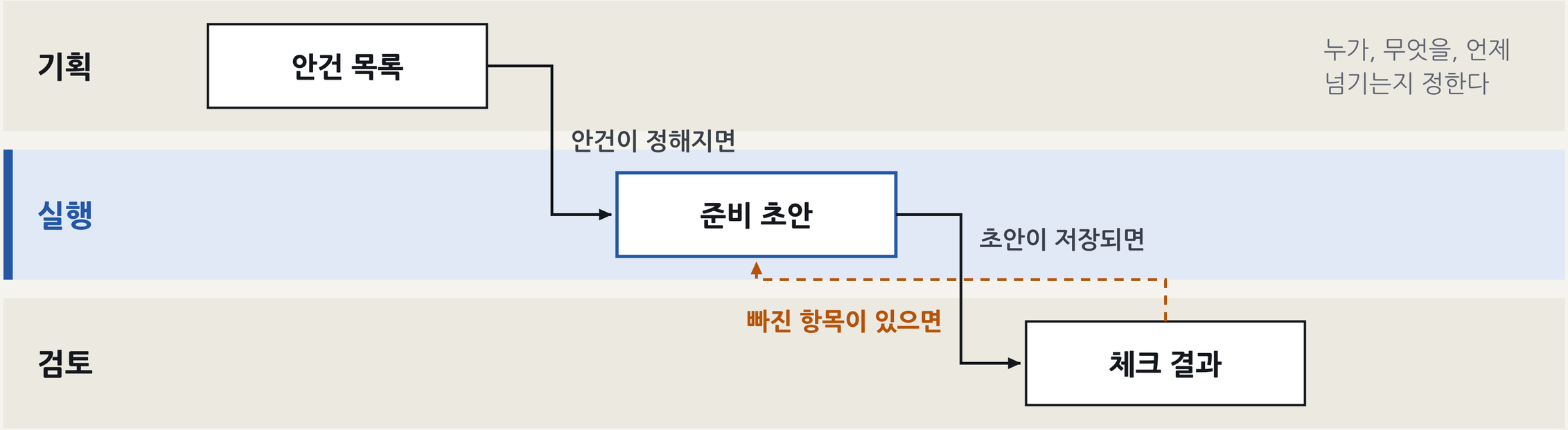
오케스트레이션

여러 단계나 실행 주체에 순서, 입출력, 역할, 다음 단계로 넘어가는 조건을 배정하고 조율하는 일.

장면

회의 준비에 안건 초안, 확인, 최종 정리가 필요하다. 맡을 사람은 아직 없다.

맡는 사람이 정해지지 않으면 같은 일을 두 번 할까?



헷갈리기 쉬운 점 역할을 나눴다고 독립 검토자가 생기지 않는다. 단일 에이전트 안의 단계 배정도 오케스트레이션이다.

도구

장면

폴더에서 최근 회의 파일 목록을 만든다.

목록을 가져오는 기능과 찾는 순서 메모, 무엇이 실행될까?

모델이 요청하고 런타임이 실행하는, 이름과 입력 형식과 결과 형식이 정해진 호출 가능한 기능.

`list_files` 도구

설명	폴더의 파일 목록을 가져온다
입력	`folder, sort`
결과	파일 이름 목록, 또는 오류

```
list_files(folder=meetings, sort=date)
```

찾는 순서 메모

1. meetings 폴더를 연다
2. 날짜순으로 정렬한다
3. 최근 파일부터 적는다

읽으면 이해되지만, 스스로 하지는 않는다

✓ **실행되는 기능. 모델이 요청하고 런타임이 실행한다.**

✕ **절차다. 실행하는 수단이 있어야 결과가 생긴다.**

헷갈리기 쉬운 점 "날짜를 ISO 형식으로 바꾼다"는 절차다. 그 변환을 실행하는 호출이 도구다. MCP는 도구를 잇는 규약이다.

스킬

특정 작업의 절차, 입력과 출력 규칙, 필요한 자원을 묶어 다시 쓸 수 있게 만든 지침 묶음.



장면

주간 고객 문의 요약을 매주 같은 순서로 만든다.

그 순서를 묶어 둔 것을 무엇이라 부를까?

입력 **새 주간 CSV와 이 묶음** → 출력 **같은 형식의 요약**

헷갈리기 쉬운 점 스킬은 도구가 아니며 스스로 실행되지 않는다. 플러그인은 스킬 등을 배포하는 포장이다.

사례 2 / 5 OMH

질문 모델만 더 좋은 것으로 바꾸면 이 일이 해결될까?

모델은 그대로 두고, 일하는 틀을 바꿨다

현장 제약

1. 오프라인 설치만
2. 로컬 파일만
3. 설정된 모델 하나

추가한 것: 업무 도구와 절차

`data_inspect`	`data_compare`	`omh-rigorous`
구조 점검	**키 기준 비교**	**선택형 절차**

유지 OpenCode와 OMO의 기존 역할

유지 설정된 단일 모델, 재학습 없음

확인 가능한 증거

- ✓ 설치와 점검 기록
- ✓ 모의 환경 테스트
- ! **사내 모델 품질은 미검증**

모의, 오프라인 점검은 기능이 연결됐다는 증거다. 실제 사내 모델의 품질을 보여 주지 않는다.

가상 예시 새 CSV → **구조 점검** 행 값 없이 → **변경** → **키 기준 비교** 자동 병합 없음

플러그인

장면

쓰기 지침, 서식 기능, 계정 연결 설정이 한 묶음으로 설치된다.

계정 연결까지 끝난 걸까?

여러 기능, 지침, 설정을 한 번에 설치하고 버전으로 관리하려고 배포하는 패키지 단위. 범위와 설치 방식은 제품마다 다르다.



헷갈리기 쉬운 점 MCP 서버 설정이 플러그인에 들어갈 수는 있지만 배포 단위와 통신 규약은 다른 층이다.

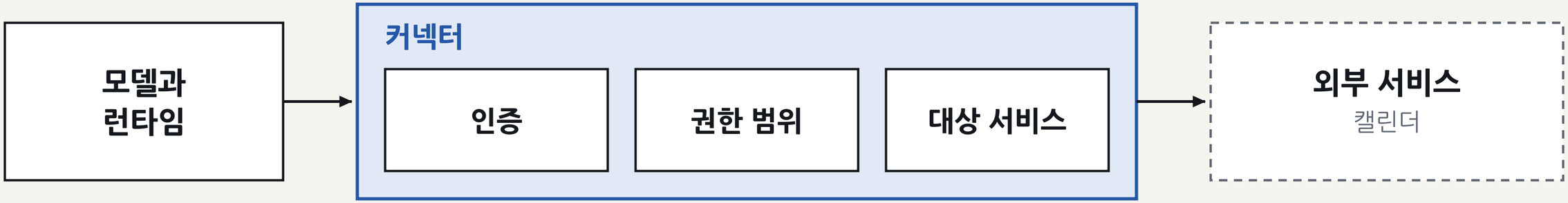
커넥터

장면

이번 주 회의 일정을 캘린더에서 가져와야 한다.

AI가 캘린더를 바로 읽을 수 있을까?

모델과 런타임이 외부 서비스, 데이터 저장소, 다른 앱의 기능에 닿도록 만든 연결 지점. 연결 설정과 인증 절차를 포함한다.



✓ 연결과 권한이 살아 있으면
일정 목록, 권한 범위만큼

✕ 연결이 없거나 만료되면
연결 오류

목록에 보이는 것과
실제로 호출되는 것은 다르다.
호출 결과로 확인한다.

헷갈리기 쉬운 점 커넥터는 외부와의 연결 지점, 도구는 모델이 호출하는 기능이다. 커넥터 하나가 여러 도구를 노출할 수 있다.

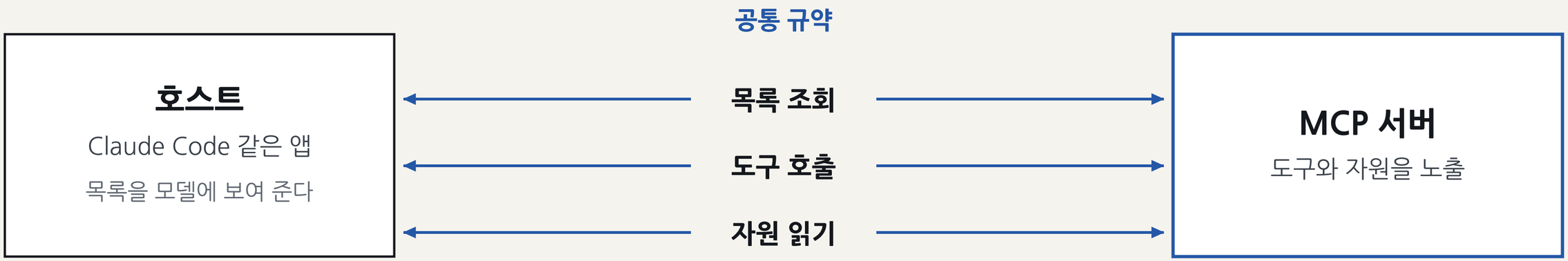
MCP

장면

여러 도구가 파일 목록과 내용을 같은 방식으로 주고받아야 한다.

도구마다 따로 약속을 만들어야 할까?

Model Context Protocol. 호스트 앱과 외부 도구, 자원 서버가 도구 목록 조회, 도구 호출, 자원 읽기를 주고받는 방식을 정한 통신 규약.



✓ **규약을 따른다**
메시지 형식과 교환 절차가 맞는다

다른 문제

✕ **접근 권한을 얻었다**
계정 연결과 허용 범위는 따로 정한다

헷갈리기 쉬운 점 모든 도구가 MCP는 아니다. 로컬 함수나 셸 명령도 MCP 없이 도구가 된다.

사례 3 / 5 Grokbot

질문 여러 봇이 역할을 나누고 공유 공간을 쓴다. 우리 팀에도 이런 구조가 맞을까?

봇의 수가 아니라, 책임과 위치를 비교한다

사용자가 설명한 패턴



사용자 설명을 그린 것. 원자료 미확인

네 개의 비교 축

1 누가 책임지는가?

Grokbot 설명	역할별 봇이 각자 책임을 나눈다
OpenClaw	코디네이터가 판단하고, 워커가 쓰고, 부모가 검증한다
CEO Agent	실무자가 반영 책임을 지고, 에이전트는 초안까지 낸다

2 작업은 어디서 실행되는가?

3 상태는 어디에 남는가?

4 사람은 어디서 개입하는가?

사용자 설명, 첨부 미확인, 패턴 비교만 한다. 제품 사양이나 성능 주장이 아니다.

03

3장 약 20분

AI는 무엇을 알고 있을까?

이 업무의 맥락을 이해하려면, 이번 호출에 실제로 들어간 정보를 확인해야 한다. 지시, 입력, 단위와 한계, 저장과 불러오기를 본 뒤 AI교양 흐름에서 확인한다.

프롬프트 — 지시와 제약

컨텍스트 — 이번 호출의 입력

토큰 — 세는 단위

컨텍스트 창 — 한 호출의 한계

메모리 — 저장과 불러오기

사례 AI교양 — 확정 게이트

프롬프트

장면

"회의 준비해 줘"와, 안건과 참석자와 지난 결정을 넣은 요청.

어느 쪽이 결과를 더 좌우할까?

모델 입력 중 지시, 질문, 역할, 제약을 담은 부분. 한 번 쓰는 요청일 수도, 여러 실행에 반복되는 지속 지침일 수도 있다.

요청 A

"회의 준비해 줘"

↓

일반적인 준비 목록

모델은 빈 곳을 추측으로 채운다

요청 B

"회의 준비해 줘"
+ 안건 세 줄
+ 참석자 명단
+ 지난 회의 결정

↓

안건 중심 준비안

입력에 없는 사실은 여전히 확인이 필요하다

헷갈리기 쉬운 점 프롬프트는 지시 문장이고, 컨텍스트는 실제로 입력된 전체다. 폴더의 규칙 파일은 읽혀야 입력이 된다.

컨텍스트

장면

지난 회의록의 경로만 알려 주고 내용은 넣지 않았다.

AI는 지난 결정을 떠올릴 수 있을까?

특정 호출에서 모델이 실제로 입력받은 모든 내용. 지시, 파일 내용, 도구 결과, 이전 대화 중 포함된 부분이 여기에 들어간다.

이번 호출에 들어간 입력

- 지시: "지난 결정 반영해서 회의 준비해 줘"
- 규칙 파일 내용
- tickets.csv 앞 20행
- 도구 결과
- 이전 대화 중 포함된 부분

디스크에만 있는 것

- 지난 회의록 내용
- `checklist.md`
- tickets.csv 21행부터

읽어서 넣어야 모델이 안다

헷갈리기 쉬운 점 메모리는 나중에 불러오려고 저장한 정보, 컨텍스트는 지금 입력에 들어 있는 정보다.

토큰

장면

같은 안내 메일을 한국어와 영어로 준비한다.

글자 수가 같으면 토큰 수도 같을까?

모델이 입력과 출력을 처리할 때 세는 텍스트 조각의 단위. 단어 수나 글자 수와 같지 않다.

한국어	안내	메일	을	보내	드립니다
영어	We	'll	send	the	notice

위 분할은 설명용 가상 예시다

글자 수가 같아도 토큰 수는 다를 수 있다

언어별 보편 비율은 없다.
정확한 수는 해당 모델의
계산 방식으로 확인한다.

헷갈리기 쉬운 점 토큰은 측정 단위이고, 컨텍스트 창은 그 단위로 잰 한계다. 이 워크숍에서는 개수를 추정하지 않는다.

컨텍스트 창

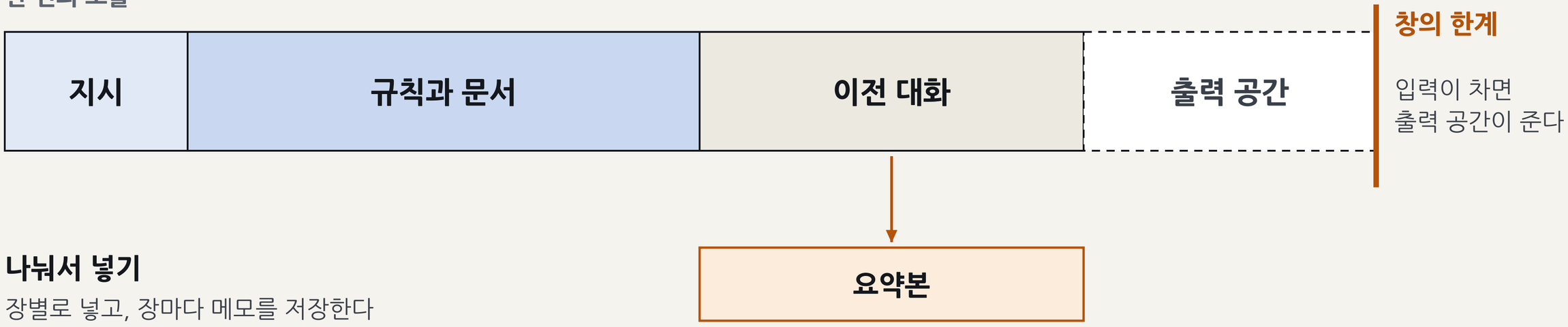
모델이 한 번의 호출에서 입력과 출력을 함께 처리할 수 있는 토큰 범위의 한계. 크기는 모델과 제품 설정마다 다르다.

장면

100쪽 계약 문서를 한 번에 넣고 검토를 맡긴다.

앞부분 내용도 끝까지 기억될까?

한 번의 호출



나눠서 넣기

장별로 넣고, 장마다 메모를 저장한다

압축: 이전 대화를 요약한다. 세부 일부가 사라질 수 있다.

헷갈리기 쉬운 점 압축은 창에 넣는 내용의 분량을 줄이는 일이다. 창의 크기는 그대로다. 메모리는 호출 밖에 저장했다가 다시 불러오는 일이다.

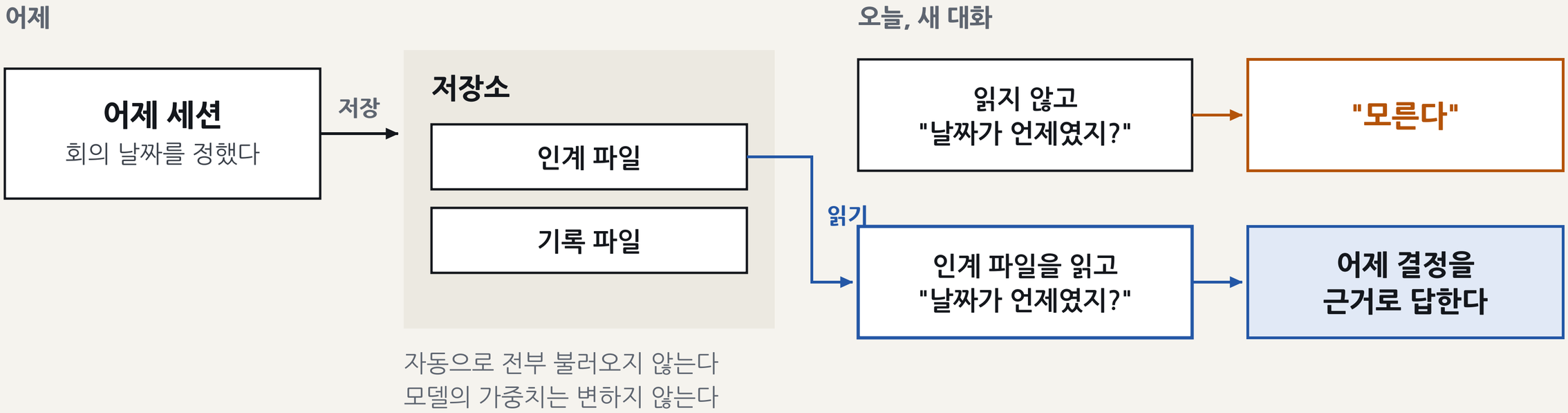
메모리

장면

어제 정한 회의 날짜를 인계 파일에 적었다.

오늘 새 대화를 열면 그 결정이 따라올까?

세션이나 실행이 끝난 뒤에도 다시 불러올 수 있게 파일이나 저장소에 남긴 정보. 모델의 가중치는 바꾸지 않는다.



헷갈리기 쉬운 점 세션이 이어진다고 메모리가 생기지는 않는다. 재학습도 아니다. 입력에 무엇이 들어가는지만 달라진다.

사례 4 / 5 AI교양

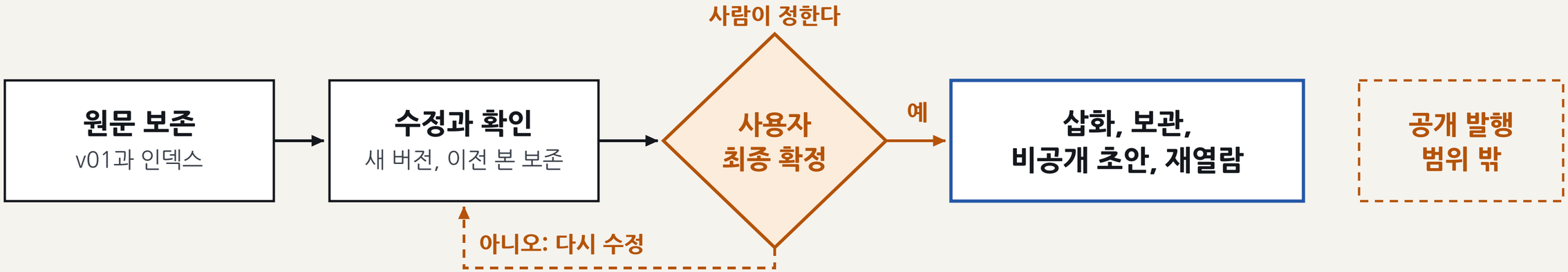
질문 최종본이 확정되기 전에 삽화부터 만들자는 요청이 왔다. 만들어도 될까?

흐름을 받치는 장치까지 보면 업무용 하네스다

규칙
원문 보존과 버전, 확정 문장 불변

컨텍스트
해당 글, 수정 이력, 관련 근거

도구
파일 저장, 이미지 생성, 초안 작성



상태
현재 버전, 확정 여부, 단계별 결과, 초안 식별자

검증
저장본 대조, 초안 재열람

완료 기록의 설명용 인용이며, 이번 작업으로 제작하지 않았다. 별도 상태 머신이 전환을 강제한다는 뜻은 아니다.

쉬는 시간

10분

돌아오면 4장

누가 이어서 일할까?

지금까지: 모델과 실행, 일하는 방법을 담는 곳, 모델이 아는 것

04

4장 약 13분

누가 이어서 일할까?

대화를 묶는 단위, 실제로 일하는 주체, 맡겨진 하위 실행을 나눈다. 그다음 한 흐름 안에서 상황별 페르소나를 고르는 사례를 본다.

세션
작업 맥락

워커
실행 담당

서브에이전트
맡겨진 하위 실행

사례 CEO Agent
상황별 선택

세션

하나의 작업 흐름을 묶어 두는 대화와 상태의 단위. 입력, 출력, 도구 기록, 진행 상태가 여기에 연결된다.

장면

같은 회의 준비 대화를 노트북과 휴대폰에서 연다.

같은 대화를 두 화면에서 보면 작업자도 두 명일까?



화면 두 개, 세션 하나

쓰기 주체는 하나여야 한다

헷갈리기 쉬운 점 세션은 실행 주체가 아니다. 세션에 저장된 이력 중 이번 호출에 들어간 부분만 컨텍스트다.

워커

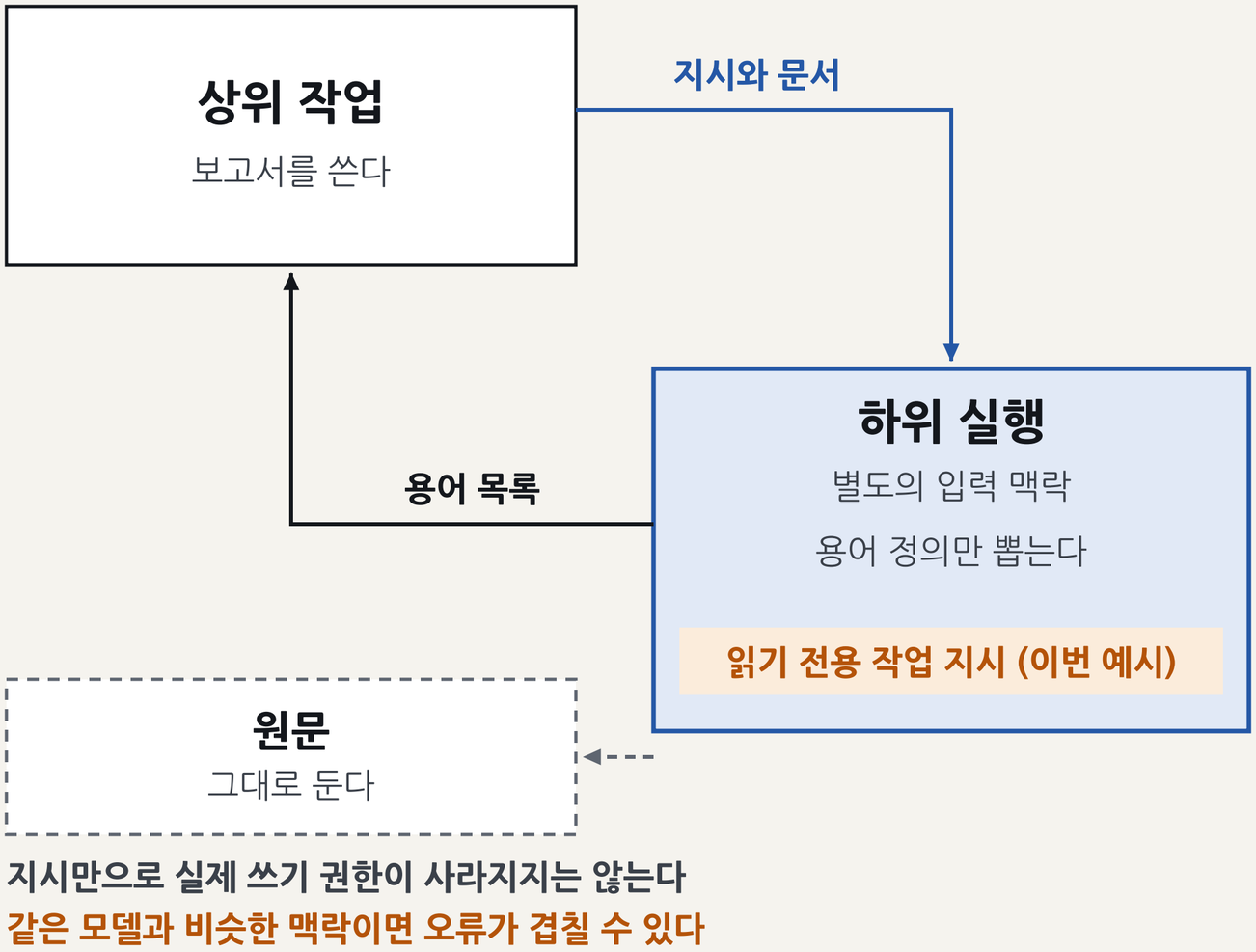
배정받은 작업을 실제로 수행하는 실행 주체. 읽기 전용 조사인지 파일 수정인지에 따라 권한과 작업 범위가 달라진다.

> **장면**
> 한 사람은 고치기만, 다른 사람은 메모만 쓰기로 나뉘었다.
> **문서를 실제로 고칠 담당은 누구로 정할까?**



헷갈리기 쉬운 점 워커는 맡은 일을 하는 역할이고, 서브에이전트는 상위와 하위의 위임 관계다. 하위 에이전트가 워커일 수 있다.

서브에이전트



상위 작업이 맡긴 하위 작업을 별도의 입력 맥락에서 수행하고 결과를 돌려주는 하위 실행. 읽기 전용일 수도, 쓰기 권한이 있을 수도 있다.

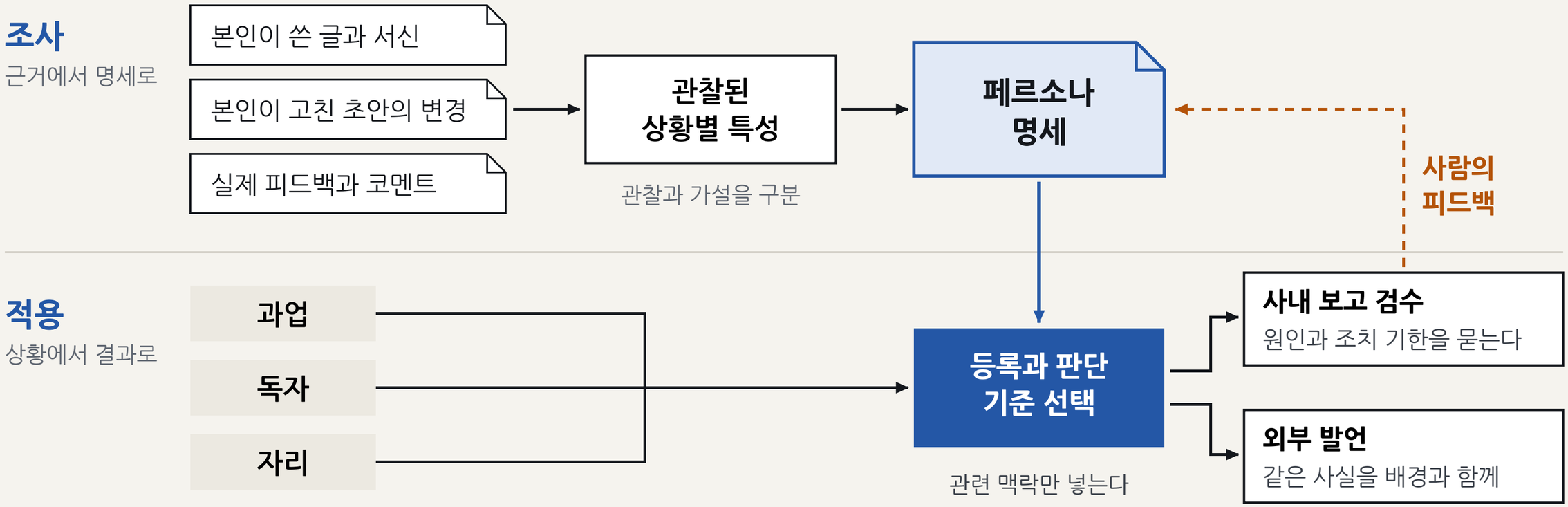
> **장면**
> 긴 문서에서 용어 정의만 뽑는 일을 나눠 준다.
> **그 작업자가 원문을 고쳐도 될까?**

헷갈리기 쉬운 점 오케스트레이션은 배정 방식이고, 서브에이전트는 실행 단위다. 서브에이전트가 있다고 독립 검토가 되지는 않는다.

사례 5 / 5 CEO Agent

질문 같은 사실이라면 사내 보고와 외부 발언의 강조도 같아야 할까?

조사한 근거로 만든 페르소나를, 상황에 맞게 골라 쓴다



같은 사실(가상): 분기 일정 지연 두 건. 원자료와 인물 정보는 공개하지 않는다. 모델을 학습시키지 않으며, 단계들은 독립된 작업자가 아니다.

05

5장 약 24분

끝났다는 말을 어떻게 믿을까?

흐름과 시작 장치, 주장과 확인을 나눈다. 일이 어디서 돌고 누가 개입하는지까지 적은 뒤, 다섯 사례를 한 틀로 되짚는다.



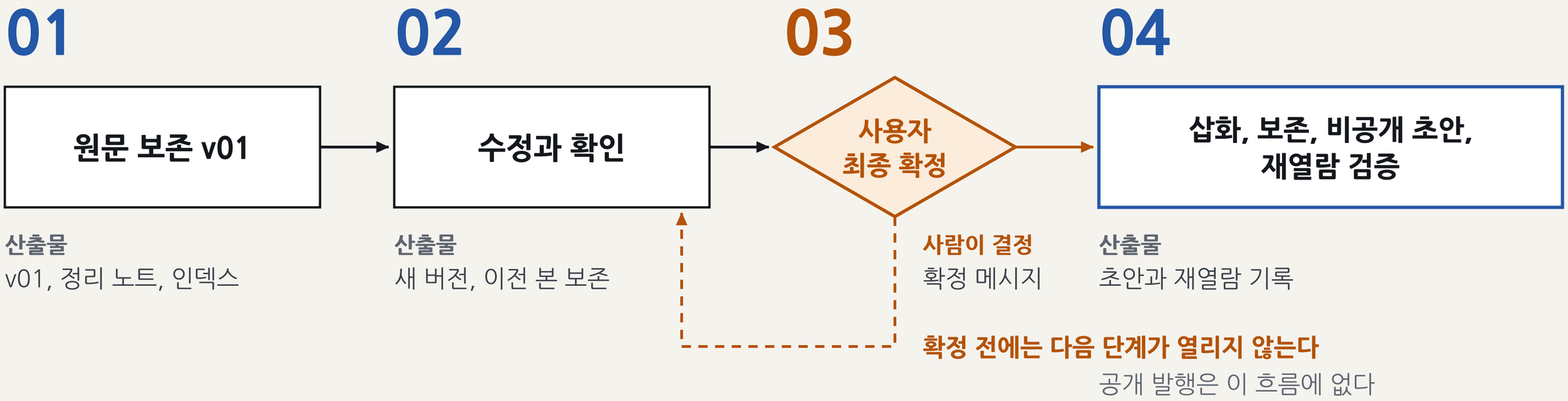
워크플로우

장면

AI교양 원문을 받았다. 최종본은 아직 확정 전이다.

삽화부터 만들어도 될까?

입력, 단계, 단계별 산출물, 다음 단계로 넘어가는 조건을 정해 둔 작업 흐름. 사람이 단계를 따라도 워크플로우다.



헷갈리기 쉬운 점 자동화는 그중 일부를 조건이 맞으면 시작하는 장치다. 하네스는 이 단계를 돌리고 검증하는 틀이다.

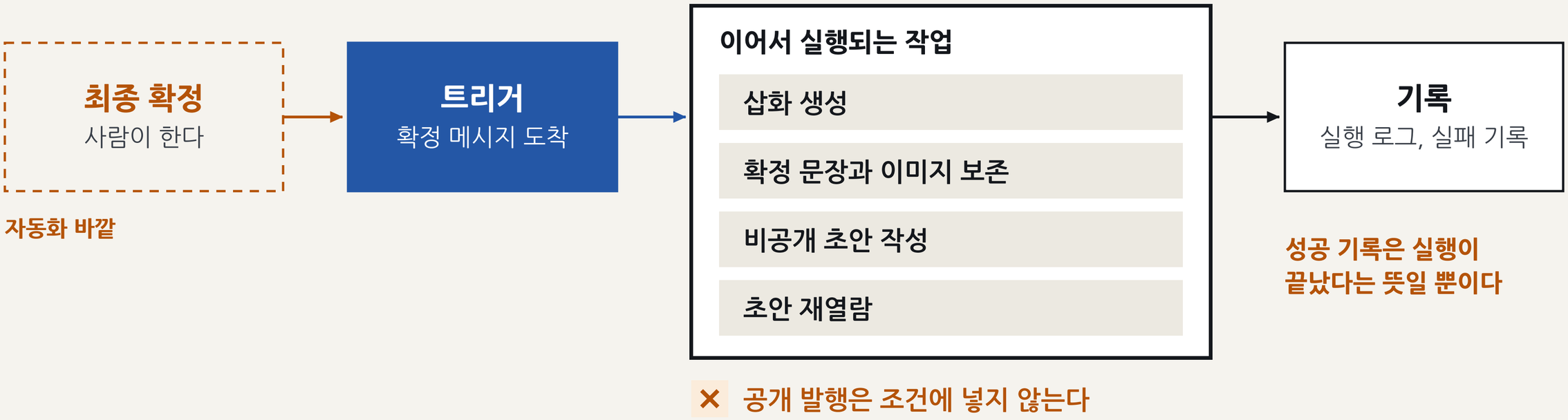
자동화

시간, 이벤트, 앞 단계 결과 같은 조건이 충족되면 사람이 다시 시작하지 않아도 정해진 작업을 실행하게 만든 장치.

장면

최종 확정 메시지가 도착했다. 저장과 초안 작성을 이어 가고 싶다.

확정 메시지 하나로 뒤의 제작 전체를 시작해도 될까?



자동화 바깥

성공 기록은 실행이 끝났다는 뜻일 뿐이다

✕ **공개 발행은 조건에 넣지 않는다**

헷갈리기 쉬운 점 자동 기억은 저장과 불러오기의 문제, 자동화는 실행 시작의 문제다. 이 장면은 설명용이며 실행하지 않았다.

완료 보고

장면

작업자가 "검토 완료" 한 줄로 보고했다.

그 한 줄이면 검토가 끝났다고 말할 수 있을까?

작업자가 "끝났다"고 말한 기록. 무엇을 했고 무엇을 확인했다고 주장하는지 담지만, 그 자체가 검증 증거는 아니다.

완료 보고	주장
상태	완료
산출물	`report.md`
확인 항목	건수 검증 완료
미확인	없음

대조 결과	확인
✓ 파일 존재	예
✗ 건수 일치	아니오
보고서 42, 원본 행 40, 고유 티켓 38	

보고는 완료, 검증은 실패. 둘을 나란히 적는다.

헷갈리기 쉬운 점 실행 도구의 done은 실행이 끝났다는 증거일 수 있지만, 요구 조건을 만족한다는 증거는 아니다.

검증

장면

주간 요약에 "이번 주 문의 42건"이라고 적혀 있다.

누가 어떻게 확인할까? 한 번 보면 충분할까?

미리 정한 기준으로 산출물, 숫자, 인용, 파일 상태, 실행 결과를 작업자의 말과 독립된 방법으로 확인해 판단하는 과정.

항목	확인 방법	결과 기록
건수 일치 결정적 확인	원본 행 수와 고유 ticket_id 수를 명령으로 세어 요약과 대조한다	**원본 40, 고유 38, 보고서 42** **실패**
모호 행 목록 결정적 확인	빈 칸이나 형식이 다른 날짜가 있는 행 번호를 뽑아 보고서와 대조한다	**빠진 행이 있으면 실패로 적는다**
근거 인용 사람의 확인	주요 주장마다 행 식별자가 인용됐는지 읽고 확인한다	아직 확인 전

기준은 작업 전에 정한다. 한 항목 통과가 전체 통과는 아니다.

헷갈리기 쉬운 점 같은 작업자가 다시 읽는 것은 약한 점검이다. 리뷰는 판단이고, 검증은 기준에 따른 확인이다.

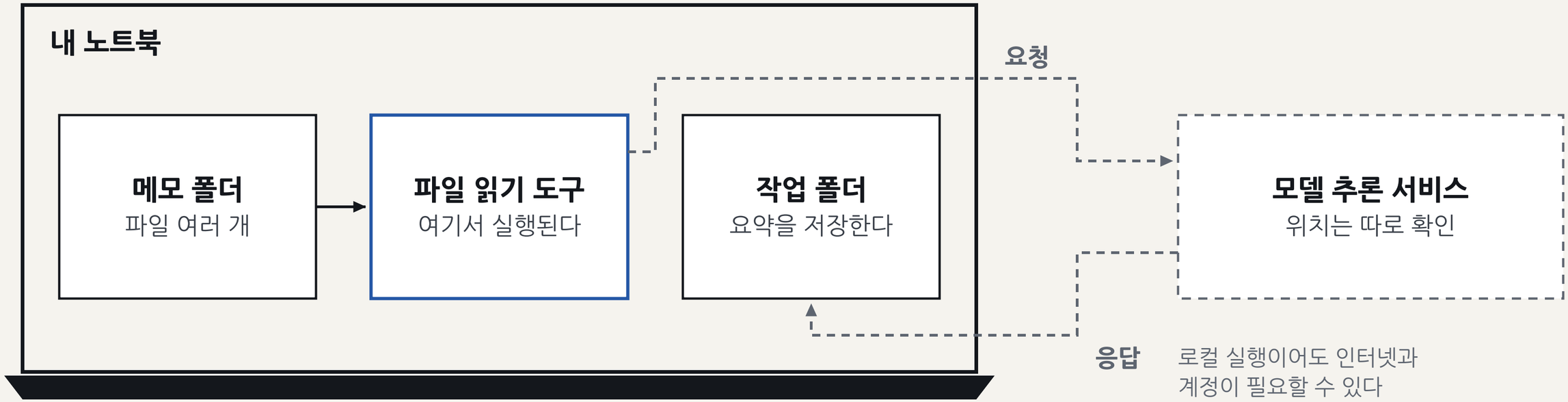
로컬 실행

장면

노트북의 메모 폴더를 AI 도구가 읽고 요약한다.

요약 답변도 노트북에서 만들어질까?

도구와 프로세스가 사용자의 컴퓨터에서 돌고, 파일 접근과 명령이 그 기기 안에서 일어나는 방식. 모델 추론 위치와는 따로 말한다.



헷갈리기 쉬운 점 "로컬에서 돌렸다"가 모델도 기기 안에서 돌았다는 뜻은 아니다. 로컬이 곧 안전하다는 뜻도 아니다.

클라우드 실행

도구, 워커, 모델 추론 중 일부나 전부가 사용자 기기가 아닌 외부 서버에서 도는 방식. 어느 구성 요소인지 함께 말해야 한다.

장면

모델은 외부 서버에서 초안을 쓰고, 파일은 노트북에 저장한다.

"클라우드 작업"이라고 한 단어로 불러도 될까?

구성 요소		내 기기	외부 서버
모델 추론	초안 텍스트를 만든다		여기
초안 파일 저장	노트북에 쓴다	여기	
파일 도구	읽고 쓰는 기능	여기	
메일 발송	이 장면에서는 하지 않는다	실행하지 않음	

"클라우드 작업"이라는 한 단어로는 어느 부분인지 알 수 없다.

헷갈리기 쉬운 점 클라우드 실행과 리모트 컨트롤은 다른 축이다. 공유 범위는 제품 설정에 따르며 전역 공유를 가정하지 않는다.

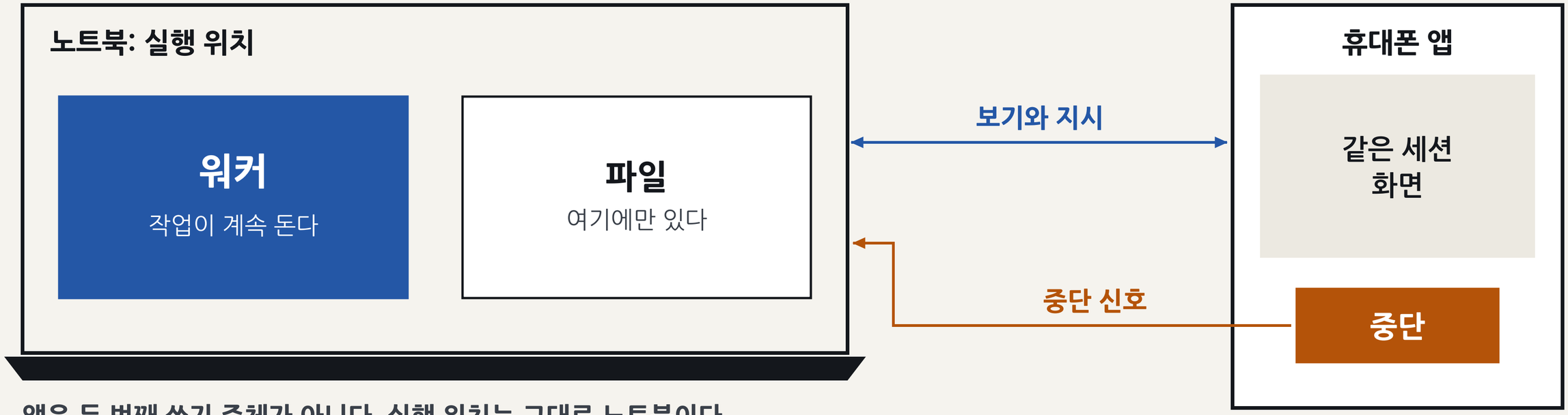
리모트 컨트롤

장면

노트북에서 도는 작업을 휴대폰에서 보고 멈추고 싶다.

멈춤을 누르면 작업 장소가 휴대폰으로 옮겨질까?

실제 실행은 원래 환경에서 계속하고, 다른 화면이나 기기에서 그 세션을 보고 지시하거나 중단 신호를 보내는 개입 방식.



앱은 두 번째 쓰기 주체가 아니다. 실행 위치는 그대로 노트북이다.

헷갈리기 쉬운 점 실행 위치와 개입 방식은 다른 축이다. 한 경로로 연결됐다고 다른 경로의 앱 가시성이 보장되지는 않는다.

종합

다섯 사례, 같은 요소, 다른 무게

규칙 | 컨텍스트 | 도구와 실행 위치 | 상태 | 검증과 사람의 최종 책임

AI교양	**흐름과 전환 조건, 사람의 확정 게이트**	재열람으로 검증
OpenClaw	**실행 위치와 개입 방식**	워커와 원격 앱
OMH	**제약에 맞춘 도구와 권한, 증거와 한계**	사내 모델 품질 미검증
Grokbot	**책임과 실행 위치의 패턴**	사용자 설명, 첨부 미확인
CEO Agent	**조사한 페르소나를 상황에 맞게 고르는 흐름**	단계는 독립 작업자가 아님

측정된 성과 비교가 아니다. 설계와 기록 구조만 비교했다.

마치며

AI에게 일을 맡길 때는 모델 바깥의 구조까지 설계한다

01	모델은 요청하고, 실행은 런타임이 한다	AI는 무엇을 하고 누가 실행할까?
02	일하는 방법은 절차와 도구로 담는다	일하는 방법을 어디에 담을까?
03	이번 호출에 실제로 들어간 정보가 컨텍스트다	AI는 무엇을 알고 있을까?
04	세션은 작업 맥락이고, 워커는 실행 담당이다	누가 이어서 일할까?
05	완료 보고는 주장이고, 검증은 확인이다	끝났다는 말을 어떻게 믿을까?

이어서 질문을 받겠습니다.

질의응답

질문을 받겠습니다

질문이 없다면: 가장 헷갈렸던 용어 하나와 그 이유는?

1장

5 모델
6 에이전트
7 런타임
8 하네스
9 **OpenClaw** ■

2장

11 오케스트레이션
12 도구
13 스킬
14 **OMH** ■
15 플러그인
16 커넥터
17 MCP
18 **Grokbot** ■

3장

20 프롬프트
21 컨텍스트
22 토큰
23 컨텍스트 창
24 메모리
25 **AI교양** ■

4장

28 세션
29 워커
30 서브에이전트
31 **CEO Agent** ■

5장

33 워크플로우
34 자동화
35 완료 보고
36 검증
37 로컬 실행
38 클라우드 실행
39 리모트 컨트롤

숫자는 슬라이드 번호다. 사각형 표시는 사례다. 확인되지 않은 답은 확인이 필요하다고 답한다.